3D Reconstruction of Structures

Jaspreet Singh Chhabra Vivek Patel

***Abstract*—We present a pipeline that takes a monocular video of a structure and reconstructs it into a navigable 3D scene. Starting from feature matching and SfM, we progressively refine the reconstruction using MVS and Gaussian Splatting. Our goal is to test how well these standard techniques hold up on real-world, imperfect data compared to controlled lab settings.**

***Index Terms*—Computer Vision, 3D Reconstruction, SfM, MVS, Gaussian Splatting**

I. Introduction

RECONSTRUCTING accurate and visually realistic three-dimensional models from two-dimensional images is a fundamental problem in computer vision and robotics. A practical application of this problem is the reconstruction of real-world structures using widely available image data, without relying on specialized hardware such as LiDAR. While LiDAR-based systems can produce highly accurate scans, they are often expensive and less accessible, making image-based reconstruction an attractive alternative.

With the increasing availability of high-quality cameras on mobile devices, it is now possible to capture large amounts of visual data with minimal effort. Converting such image or video data into detailed 3D models enables applications in visualization, simulation, and scene understanding. However, unlike controlled datasets, real-world captures introduce challenges such as motion blur, lighting variation, and limited viewpoint diversity, which can significantly impact reconstruction quality.

In this project, we focus on reconstructing structures on the University of Alberta campus using videos recorded with a mobile device. The goal is to extend core computer vision techniques to a more realistic and complex scenario. To achieve this, we design an end-to-end pipeline that begins with feature extraction using SIFT, enhanced through techniques such as CLAHE. Feature correspondences are then established using different matching strategies, followed by incremental Structure-from-Motion (SfM) using COLMAP to recover camera poses and sparse reconstruction. This is further refined using Multi-View Stereo (MVS) to obtain a dense reconstruction, and finally represented using Gaussian Splatting for improved visual quality and real-time rendering.

Through experiments on multiple datasets, we systematically evaluate key components of the pipeline to understand how errors propagate through real-world settings. We study feature extraction choices, such as SIFT parameter tuning and preprocessing (e.g., CLAHE), and how they impact keypoint detection. For correspondence, we compare sequential brute-force matching against temporal window-based approaches with KD-Tree acceleration, analyzing their effect on match density and track continuity. Furthermore, we examine the influence of camera settings and radial distortion on reconstruction stability. Finally, we analyze the role of outlier rejection techniques—such as Lowe's ratio test and RANSAC—alongside point cloud noise reduction using statistical outlier removal and DBSCAN. These components are ultimately evaluated based on their impact on sparse reconstruction quality, completeness, and downstream Gaussian Splatting results.

Email: jchhabr1@ualberta.ca
Email: vivekra1@ualberta.ca

Fig. 1: Dense Reconstruction of Computing Science Center, University of Alberta

II. Related Works

Feature-based methods form the foundation of image-based 3D reconstruction pipelines. The Scale-Invariant Feature Transform (SIFT) proposed by Lowe [1] detects keypoints as extrema in scale-space using the Difference of Gaussians (DoG), defined as

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma), \tag{1}$$

where $L(x, y, \sigma)$ is the image convolved with a Gaussian kernel at scale σ. The resulting descriptors are invariant to scale and rotation, making them suitable for matching across views. In practice, real-world images often suffer from poor contrast, and preprocessing techniques such as CLAHE improve local contrast and increase feature detectability [2].

Feature matching plays a critical role in reconstruction quality. Standard approaches include brute-force matching and KD-tree-based nearest neighbor search, with correspondences filtered using Lowe's ratio test and RANSAC to remove outliers [3]. These steps improve geometric consistency and reduce the impact of incorrect matches.

Given feature correspondences, Structure-from-Motion (SfM) estimates camera poses and reconstructs a sparse 3D point cloud. A key geometric relation is the epipolar constraint

$$\mathbf{x}'^{T}\mathbf{F}\mathbf{x} = 0, \tag{2}$$

where $\mathbf{F}$ is the fundamental matrix relating corresponding points $\mathbf{x}$ and $\mathbf{x}'$. Incremental SfM pipelines, such as COLMAP

[4], [5], iteratively register images and triangulate 3D points by minimizing the reprojection error

$$\min_{\mathbf{X}} \sum_{i} \|\mathbf{x}_i - \pi(\mathbf{P}_i \mathbf{X})\|^2, \tag{3}$$

where $\mathbf{P}_i$ is the projection matrix of camera i and $\pi(\cdot)$ denotes perspective division.

To obtain dense geometry, Multi-View Stereo (MVS) extends sparse reconstructions by enforcing photometric consistency across multiple views. This is commonly formulated as minimizing intensity differences between corresponding pixels:

$$\min_{d} \sum_{i,j} \|I_i(\mathbf{x}) - I_j(\mathbf{x}')\|^2, \tag{4}$$

where $\mathbf{x}'$ is the projection of a point at depth d into another view. While effective, MVS is sensitive to low-texture regions and errors in feature matching.

Recent work such as Gaussian Splatting [6] represents scenes using a collection of anisotropic 3D Gaussians. Each Gaussian is parameterized by its mean $\boldsymbol{\mu} \in R^3$, covariance $\boldsymbol{\Sigma} \in R^{3\times3}$ (encoding scale and orientation), color $\mathbf{c}$, and opacity α, resulting in a compact representation of both geometry and appearance. Rendering is performed via alpha compositing:

$$C = \sum_{i} w_i c_i, \quad w_i = \alpha_i \prod_{j<i} (1 - \alpha_j), \tag{5}$$

allowing efficient real-time rendering with view-dependent appearance.

III. Methodology

A. Data Collection

For this project, we evaluated our reconstruction pipeline using three video datasets to understand how it handles varying scales and textures. While differences in scale were manageable, variations in texture and feature richness proved more challenging for stable feature detection and matching. Our primary target was the Computing Science Centre (CSC) building; however, due to its complexity, we first validated our pipeline on simpler objects: a small toy house and a feature-rich jet engine model. As shown in Fig. 2, these datasets span a range of scales and geometric complexity.

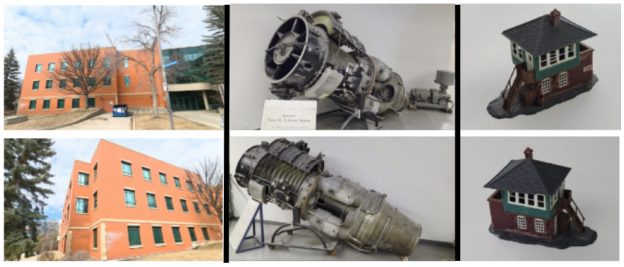

Fig. 2: Data collection across three datasets: CSC (left), jet engine (center), and toy house(right)

Earlier recordings of the CSC building were captured in portrait orientation, which led to significant issues during reconstruction. Due to the limited field of view, portions of the building were frequently clipped at the image boundaries during camera motion, breaking feature tracks and reducing correspondence consistency. This resulted in fragmented reconstructions, producing multiple disjoint point clouds instead of a single coherent model, as shown in Fig. 3.

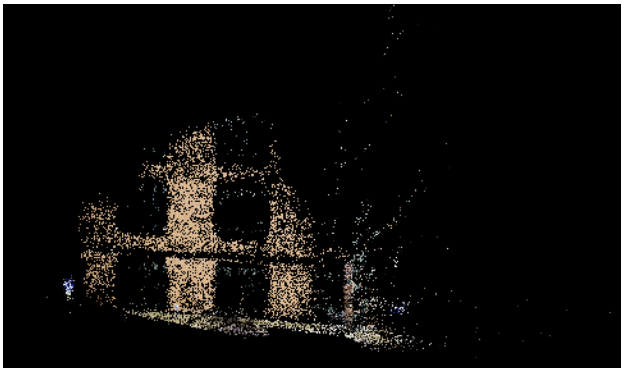

Fig. 3: Failure case for CSC reconstruction: portrait capture and incomplete field-of-view lead to broken feature tracks and multiple disjoint point clouds.

To address this, we switched to landscape capture with a wider field of view. However, the large scale of the building and constrained campus viewpoints required the use of an ultra-wide lens to fully capture the structure. While this resolved the clipping issue, it introduced significant radial distortion that required correction. In contrast, the toy house and jet engine datasets were captured using a standard lens with minimal distortion, leading to more stable reconstructions.

Lens	MP	Focal Length	Aperture / Sensor
Wide (Main)	50 MP	24 mm	f/1.8, 1/1.56”, 1.0µm
Ultra-wide	12 MP	13 mm	f/2.2, 1/2.55”, 1.4µm

TABLE I: Camera specifications for the Samsung Galaxy S25 used during data collection [7].

All videos were recorded at 1080p and 30 FPS using a Samsung S25. We used both the primary wide and ultra-wide lenses depending on scene constraints, with their specifications summarized in Table I. We later standardized capture to landscape orientation to preserve the full scene during motion and avoid feature loss at image boundaries. To maintain consistency across frames, automatic exposure and autofocus were locked, preventing feature drift caused by dynamic changes in brightness or focal length. Finally, videos were subsampled to 2–5 FPS to ensure sufficient baseline for triangulation, and blurry frames were automatically filtered using Laplacian variance.

B. Feature Extraction

We initially used SIFT with default parameters for feature extraction; however, this resulted in poor reconstruction quality on the CSC dataset (see Fig. 3). The primary issue was the lack of sufficient feature points on large, low-texture regions such as the brick walls of the building, which led to weak geometric constraints during matching and triangulation.

Fig. 4: SIFT keypoints: default (left), edge + lower contrast (center), CLAHE (right).

To address this, we experimented with tuning key SIFT parameters, particularly `contrastThreshold` and `edgeThreshold`. Lowering the contrast threshold allowed weaker features to be detected, while adjusting the edge threshold increased sensitivity near structural boundaries. This resulted in approximately a $2\times$ increase in detected keypoints, as summarized in Table II. However, the overall reconstruction quality did not improve significantly. While more features were detected, many of the additional points were concentrated along edges, which are inherently unstable for matching due to the aperture problem.

Feature Extractor	Configuration	Avg. Keypoints
SIFT (Default)	Default parameters	2300
SIFT (Modified)	Lower contrast + edge	6000
SIFT (CLAHE)	CLAHE + preprocessing	25000

TABLE II: Average number of detected SIFT keypoints on the CSC dataset under different configurations.

To overcome this limitation, we incorporated Contrast Limited Adaptive Histogram Equalization (CLAHE) as a preprocessing step. CLAHE enhances local contrast, making previously indistinct regions more feature-rich. This led to a substantial increase in detected keypoints (approximately $10\times$ compared to default SIFT), with features now distributed more uniformly across surfaces such as the CSC brick walls. As shown in Fig. 4, this preprocessing step significantly improved feature coverage. This improvement also translated to stronger and more numerous correspondences, as illustrated in Fig. 5.

C. Feature Matching

To ensure reliable correspondences, we filtered incorrect matches using both appearance- and geometry-based constraints. First, Lowe's ratio test [8] was applied to remove ambiguous matches by comparing the distances of the nearest and second-nearest neighbors in descriptor space. This eliminates ambiguous matches arising from repetitive or low-texture regions. Next, RANSAC [9] was used to enforce geometric consistency by estimating a robust transformation between frames and discarding outliers that do not satisfy the model. As shown in Fig. 5, this two-stage filtering reduces the initial set of matches to a smaller set of high-quality inliers.

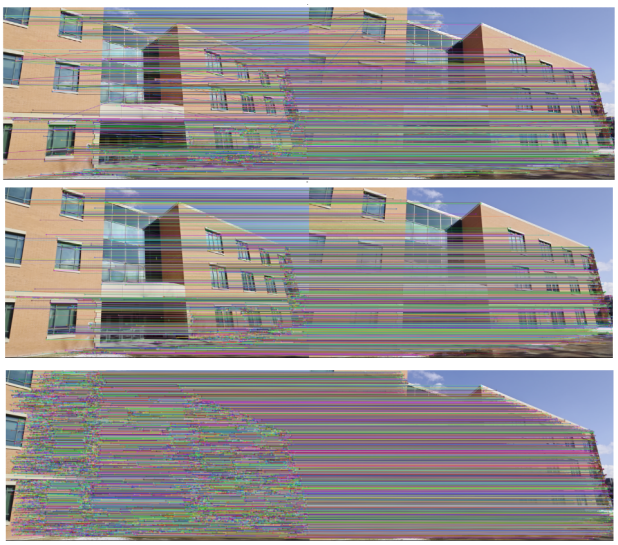

Fig. 5: Feature matching results (top to bottom): SIFT ($\sim$2500), filtered inliers ($\sim$1900), and CLAHE-enhanced matches ($\sim$15000).

Once reliable features were extracted, the next step was to establish correspondences across frames. We initially employed a standard sequential brute-force matcher, where each frame t is matched only with frame $t+1$. While simple, this approach proved to be highly fragile in practice. Even minor occlusions, motion blur, or temporary feature loss would break tracking chains, leading to inconsistent correspondences. As a result, the reconstruction pipeline failed to produce a unified model of the CSC building, instead generating multiple fragmented point clouds (see Fig. 3).

Addressing this limitation required us to adopt a temporal window matching strategy. Instead of restricting matches to adjacent frames, each frame was matched against a local temporal window (e.g., $t\pm10$ frames). This created a redundant network of feature correspondences, allowing tracks to persist even when features were temporarily occluded or lost in intermediate frames. This approach proved particularly effective for the CSC dataset, where occlusions from trees and structural complexity frequently disrupted sequential matching.

Since such a window-based matching strategy significantly increases computational cost, we optimized the matching process by indexing SIFT descriptors using a KD-Tree. This enabled efficient approximate nearest neighbor (ANN) search

in the 128-dimensional descriptor space, reducing the practical complexity from $\mathcal{O}(N^2)$ to approximately $\mathcal{O}(N \log N)$. This optimization made the approach computationally feasible while retaining robustness.

D. Sparse Reconstruction

For the actual Structure-from-Motion (SfM) step, we used COLMAP [4], [5]. Having utilized a custom KD-Tree matching pipeline to solve the sequential tracking failures, we were required to bypass COLMAP's default extraction step. Consequently, we engineered a script to programmatically inject our verified matches directly into the COLMAP SQLite database. This populated database then seeded COLMAP's Incremental Mapping engine to handle pose registration, triangulation, and bundle adjustment. The resulting sparse reconstruction and estimated camera trajectory for the CSC dataset are shown in Fig. 6.

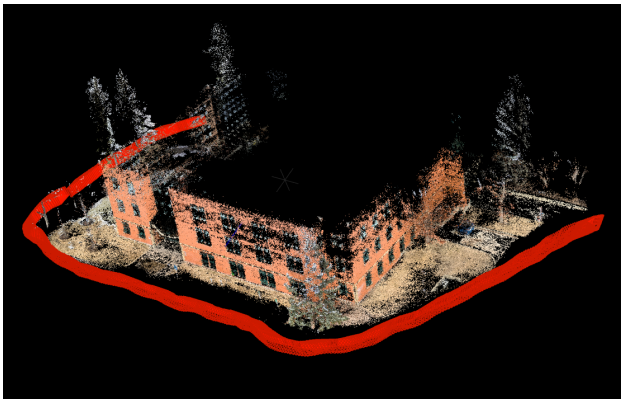

Fig. 6: Sparse Reconstruction of CSC Dataset; the red frustums represent the camera path found using COLMAP

Our biggest hurdle here was dealing with the extreme lens curvature from the ultra-wide camera we used for the CSC dataset. Initially, the reconstructed building physically bowed inward. To fix this, we initialized the bundle adjustment with the OPENCV camera model, which forced the solver to calculate the specific radial distortion coefficients, (k_1, k_2). Once it optimized those values, we were able to unwarp the geometry into a strict PINHOLE projection, successfully snapping the building's corners back to 90 degrees, as illustrated in Fig. 7.

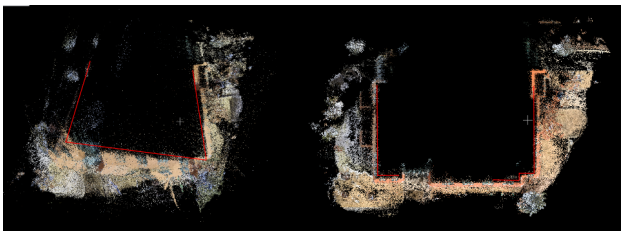

Fig. 7: Before and After Radial Distortion Correction on CSC Dataset

Lastly, the raw output from COLMAP needed some cleanup. Triangulation naturally creates floating artifacts in unconstrained background areas like the open sky. We applied Statistical Outlier Removal (SOR) and DBSCAN clustering directly to the .bin files to mathematically filter out this disconnected noise. This reduced the point cloud size by 2.1% and improved our overall geometric accuracy, dropping the mean reprojection error from 0.426 px to 0.421 px. Cleaning up the point cloud like this is also a huge help if we want to use 3D Gaussian Splatting later, since leaving background noise in usually causes the algorithm to fill empty space with massive floating artifacts. The effect of this cleanup process is shown in Fig. 8.

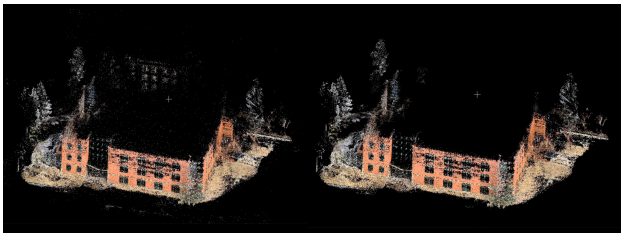

Fig. 8: Before and After Noise Removal using Statistical Outlier Removal and DBSCAN on CSC Dataset

E. Densification

1) Multi-View Stereo (MVS): To convert the sparse SfM reconstruction into a dense representation, we used COLMAP's PatchMatch-based Multi-View Stereo (MVS) pipeline. This stage estimates per-pixel depth by enforcing photometric consistency across multiple views, effectively reconstructing fine-grained surface details that are not captured during sparse triangulation.

Specifically, PatchMatch computes depth maps for each image by iteratively propagating depth hypotheses across neighboring pixels, followed by a consistency check across views. These depth maps are then fused into a single dense point cloud using COLMAP's stereo fusion module, ensuring geometric consistency across all views. The resulting dense reconstruction (Fig. 1) represents a significant improvement over the sparse model, with substantially higher point density and improved surface coverage.

To further convert the dense point cloud into a continuous surface, we experimented with Poisson surface reconstruction [10]. This method fits an implicit surface to the point cloud by enforcing consistency with estimated normals, producing a smooth and watertight mesh.

While effective in theory, Poisson reconstruction did not perform well on our CSC dataset. In regions with low point density or weak geometric constraints, the method tends to over-smooth and hallucinate surfaces, filling gaps with incorrect geometry. As shown in Fig. 9, this results in noticeable artifacts and loss of structural detail, particularly in under-constrained areas.

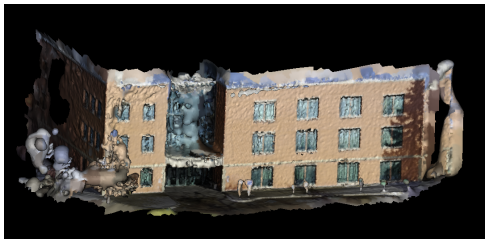

Fig. 9: Poisson Meshing on CSC Dataset

2) Gaussian Splatting: The core objective of the dense reconstruction phase was to achieve a photorealistic scene representation using 3D Gaussian Splatting (3DGS) [6]. However, 3DGS is highly VRAM-intensive, and processing the full dataset of approximately 320 high-resolution images would exceed the memory limits of the 15GB Colab and Kaggle instances used in this project. To mitigate this, we downsampled the input images by a factor of 1.2 prior to training.

To initialize the 3DGS model, we used the cleaned sparse point cloud. As discussed in the previous section, providing this accurate geometric base constrained the optimization from the first iteration, effectively preventing the "floater problem" (Fig. 10). Over 30,000 iterations, the model optimized the position, covariance, and opacity of a set of 3D Gaussians. In addition to geometry, it also learned view-dependent appearance using Spherical Harmonics, enabling realistic rendering under varying viewpoints. This was particularly important for the CSC dataset, where it allowed accurate reconstruction of reflective surfaces such as glass facades.

The final Gaussian Splat reconstructions for the toy house, jet engine, and CSC datasets are shown in Fig. 13, Fig. 14, and Fig. 15, respectively, demonstrating high-quality photorealistic rendering across varying scales and scene complexities.

Fig. 10: "floater problem" in a Gaussian Splat

IV. Results

We evaluate our success along two main objectives: (1) completion of both planned and extended pipeline components, and (2) geometric and visual accuracy of the reconstructed scenes.

From a pipeline perspective, all core components were successfully implemented within the project timeline. Beyond the baseline Structure-from-Motion pipeline, we achieved our extended goals, including dense reconstruction via Multi-View Stereo (MVS), Poisson surface reconstruction, and a comparative study of feature extraction strategies.

From a qualitative standpoint, the reconstructions demonstrate strong visual correctness, with consistent geometry and well-preserved structural properties such as orthogonality of edges and planar surfaces. This is particularly evident in the CSC building, where architectural features such as windows and façade boundaries remain well-aligned in the final reconstruction.

To quantitatively evaluate geometric accuracy, we performed real-world measurements on both the toy house and CSC datasets and compared them against measurements obtained from the reconstructed Gaussian Splat models. For the toy house, this process was straightforward due to its small scale and accessibility, allowing direct measurement of multiple sides and features (Fig. 12). In contrast, for the CSC building, measurements were limited to observable features such as window dimensions (Fig. 11), which provided a reliable reference for evaluating reconstruction accuracy.

Since the reconstructed models are defined up to an unknown scale, a global scale factor was estimated for each dataset before computing errors. After scale alignment, the results show low reconstruction error across both datasets. For the CSC dataset, the window measurements exhibit errors of approximately 2–3%, while the toy house measurements show errors consistently below 4%, as summarized in Tables III and IV. These results indicate that the pipeline achieves good metric consistency despite challenges such as lens distortion and limited feature regions.

Measurement	Real (cm)	Splat (cm)	Error (cm)	Error (%)
Window Width	160	156.57	-3.43	2.14
Window Height	198	202.62	+4.62	2.33

TABLE III: Scale-corrected measurements for CSC Dataset using a global scale factor of 9.21.

Measurement	Real (cm)	Splat (cm)	Error (cm)	Error (%)
Roof Side 1	7.00	7.13	+0.13	1.86
Roof Side 2	8.00	7.70	-0.30	3.75
Window Width	2.30	2.27	-0.03	1.30
Body Side 1	5.20	5.18	-0.02	0.38

TABLE IV: Scale-corrected measurements for ToyHouse Dataset using a global scale factor of 0.081.

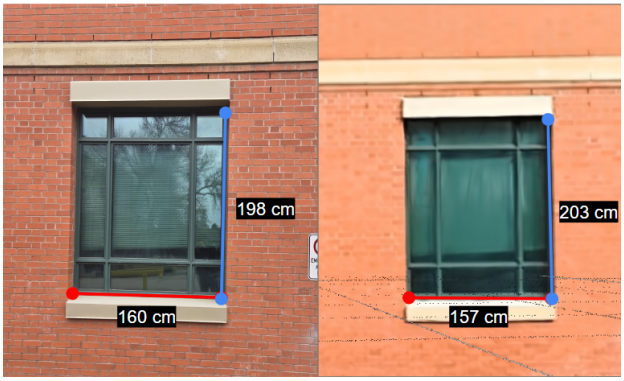

Fig. 11: Real vs reconstructed (splat) measurements of the CSC Dataset

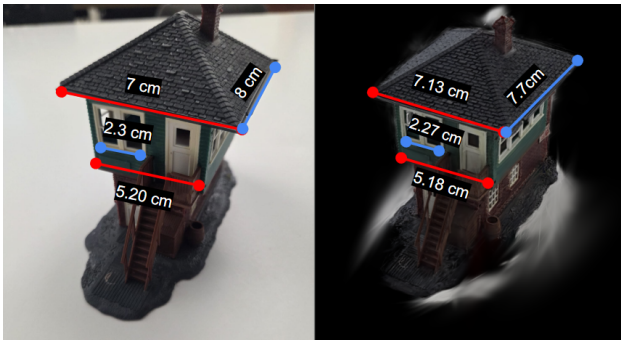

Fig. 12: Real vs reconstructed (splat) measurements of the Toy House Dataset

Finally, the reconstructed Gaussian Splat models for all datasets are shown in Fig. 13, Fig. 14, and Fig. 15. These results demonstrate that the proposed pipeline is capable of producing high-quality, photorealistic reconstructions across a range of scenes, from small-scale objects to large architectural structures.

All outputs are available at this link, with code on GitHub at this link.

Full-resolution visualizations are provided in the Appendix (Sec. 16).

Fig. 13: Gaussian Splat for Toy House Dataset

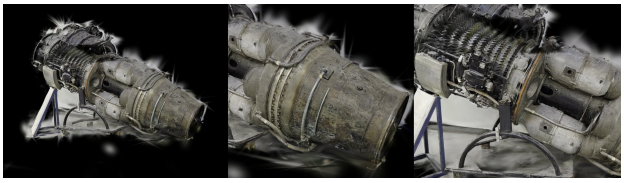

Fig. 14: Gaussian Splat for Jet Engine Dataset

Fig. 15: Gaussian Splat for CSC Dataset

ACKNOWLEDGMENT

We would like to thank our professor Martin Jagersand and our TAs Riley Zilka and Cole Dewis for their support and guidance throughout this project. We also acknowledge the use of large language models (LLMs), including ChatGPT, which were used to assist with code development, debugging, and the refinement of writing in this report.

REFERENCES

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[2] *Histogram equalization*. [Online]. Available: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html.

[3] *Feature matching*. [Online]. Available: https://docs.opencv.org/3.4/dc/dc3/tutorial_py_matcher.html.

[4] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.

[5] J. L. Schonberger, *Colmap*. [Online]. Available: https://github.com/colmap/colmap.

[6] B. Kerbl, G. Kopanas, T. Leimkuhler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, 2023.

[7] GSMArena, *Samsung galaxy s25 specifications*, https://www.gsmarena.com/samsung_galaxy_s25-13610.php, 2025.

[8] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[9] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[10] M. Kazhdan, M. Bolitho, and H. Hoppe, "Poisson surface reconstruction," in *Proceedings of the Eurographics Symposium on Geometry Processing*, 2006.

APPENDIX

To improve clarity, we provide full-resolution versions of key intermediate and final outputs referenced throughout the report.

A. Sparse Reconstruction

B. Dense Reconstruction (MVS)

C. Radial Distortion Correction

D. Noise Removal

E. Final Gaussian Splat Results

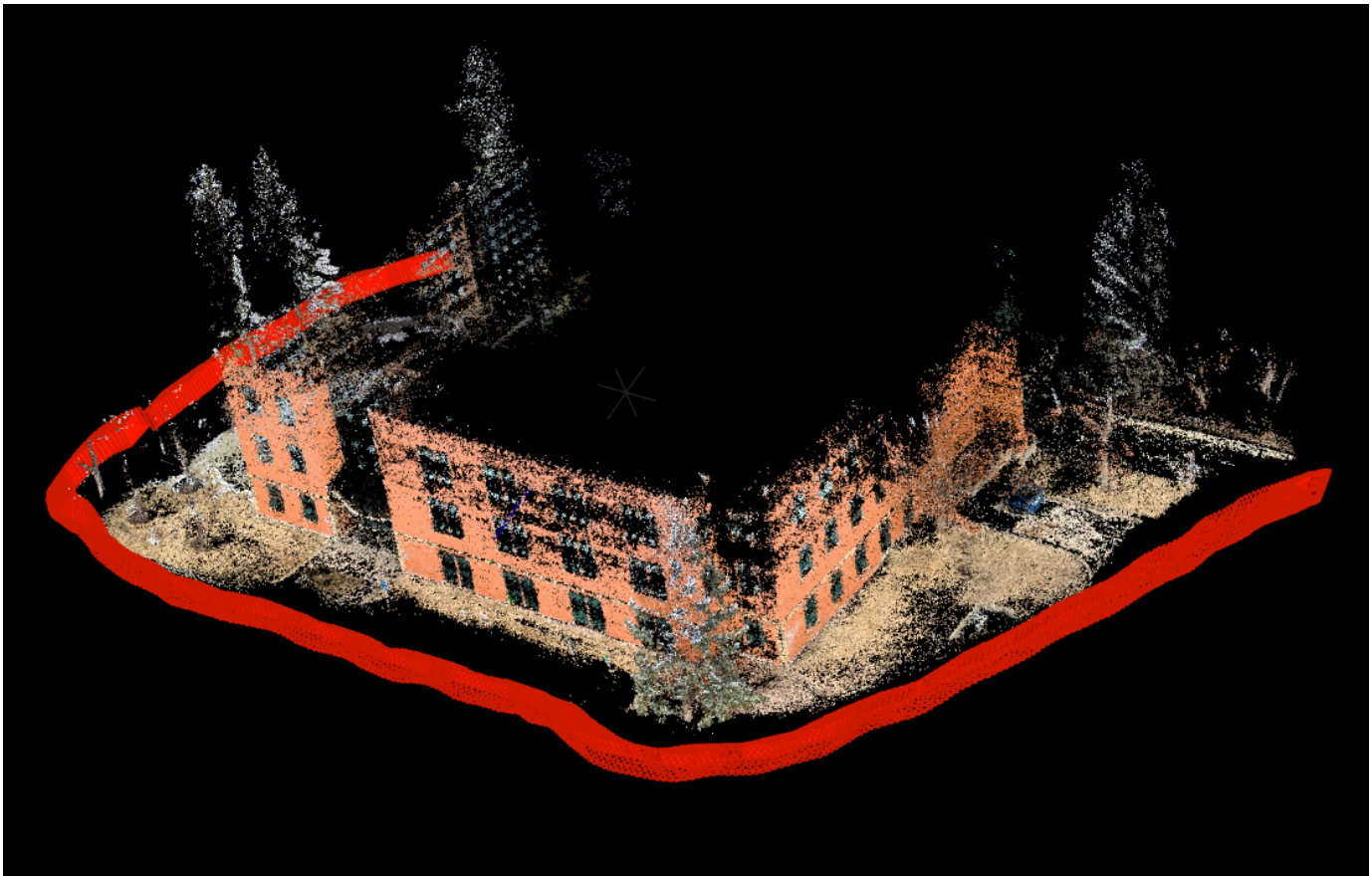

Fig. 16: Sparse Reconstruction of the CSC dataset.

Fig. 17: Dense Reconstruction (MVS) of the CSC dataset.

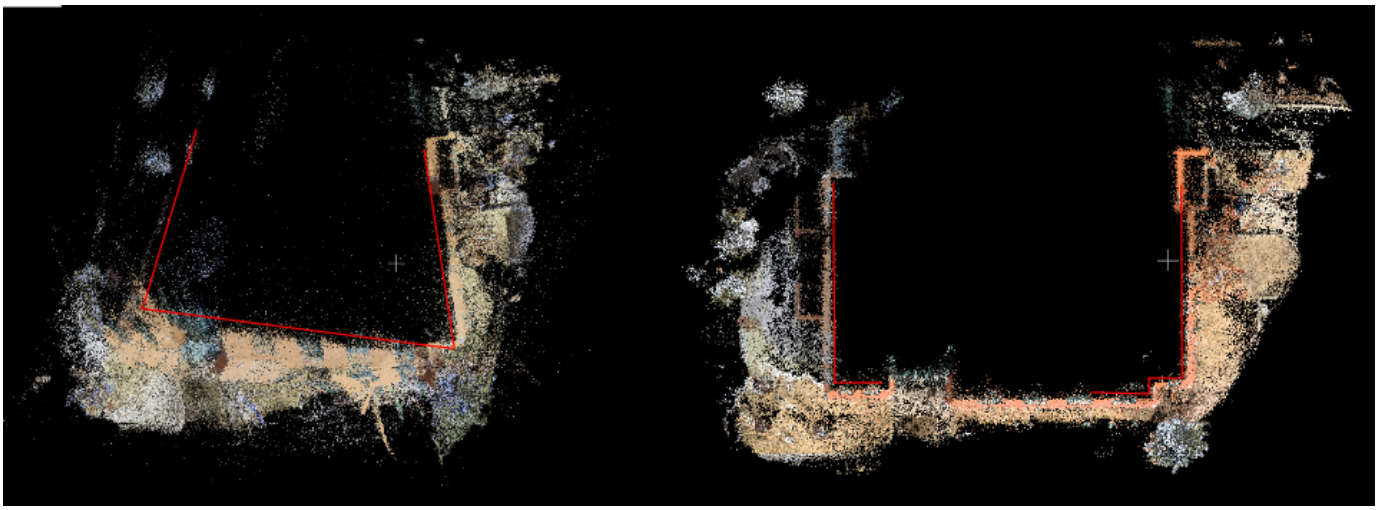

Fig. 18: Comparison before and after radial distortion correction.

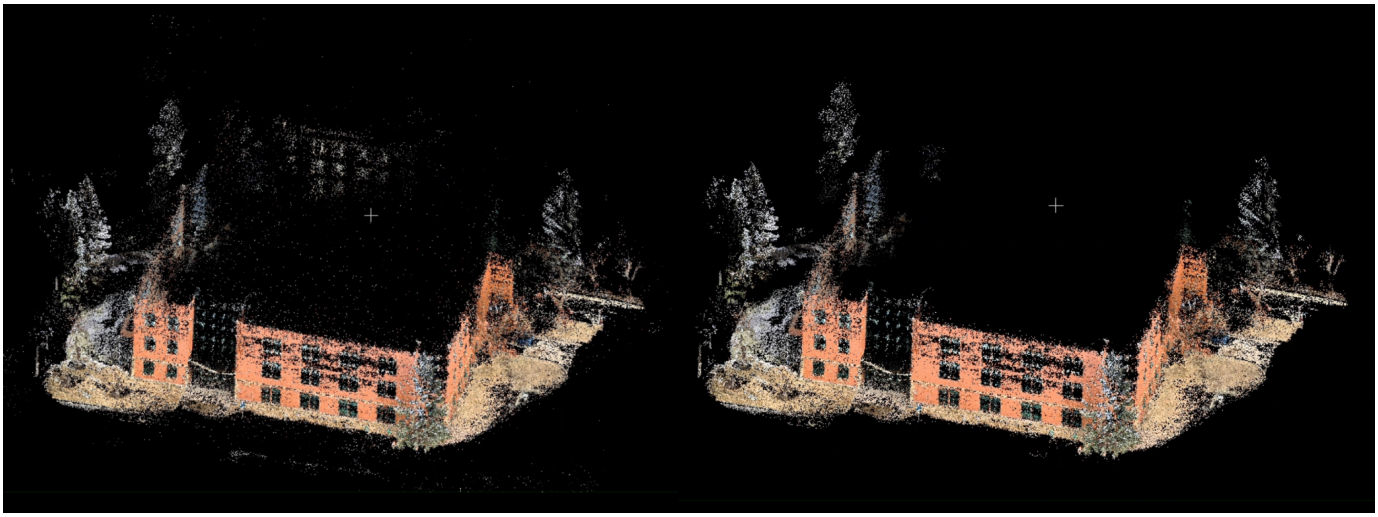

Fig. 19: Comparison before and after noise removal using SOR and DBSCAN.

Fig. 20: Gaussian Splat result for the Toy House dataset.

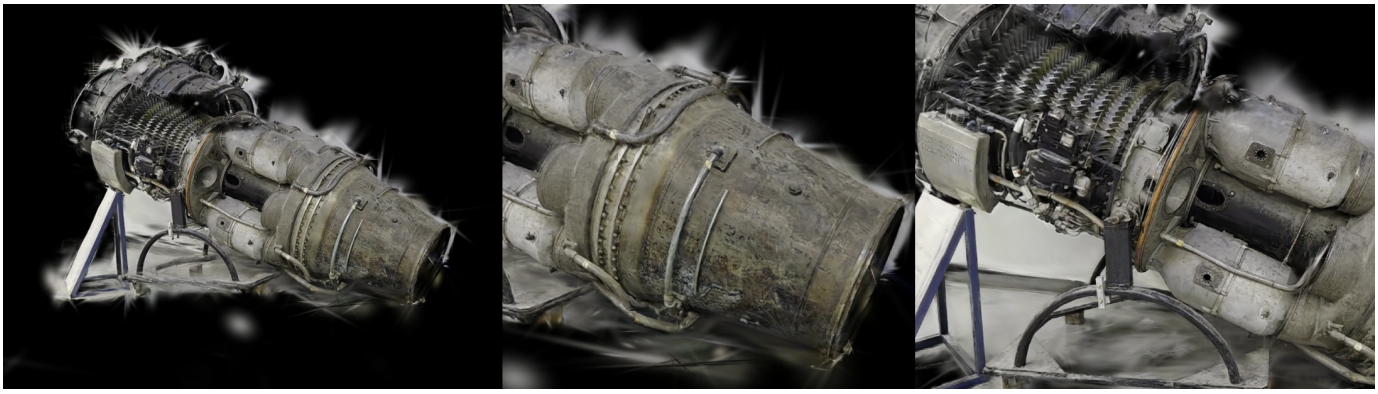

Fig. 21: Gaussian Splat result for the Jet Engine dataset.

Fig. 22: Gaussian Splat result for the CSC dataset.